TRANSPORT AND TELECOMMUNICATION INSTITUTE

ENGINEERING FACULTY

Practical work N1

Course: **Computer Systems Structure**

Theme: **The Von Neumann Machine**

Student: Igors Oļeiņikovs

Student code: 93642

Group: 4501BTA

Riga
2026

CONTENTS

1. EXERCISE 1

Exercise 1: *Understanding the vnmsim Interface*

Objective: Familiarize with the vnmsim platform and identify components of the Von Neumann architecture.

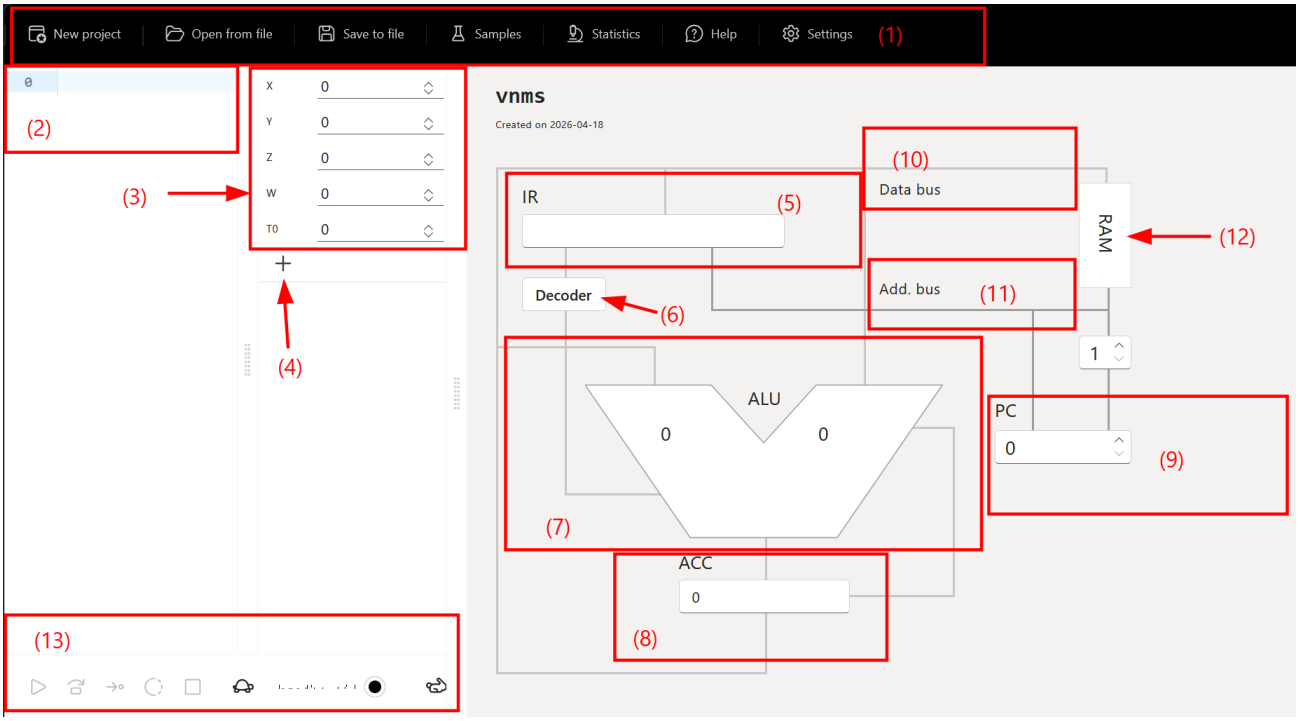

Fig. 1 Initial vnmsim interface

Main elements visible in the simulator

Interface element	**Role in vnmsim**	**Von Neumann architecture meaning**
Top menu **(1)**	Gives access to New project, Open from file, Save to file, Samples, Statistics, Help, and Settings	These controls do not belong to the theoretical machine itself; they are the environment used to load programs and inspect execution

RAM editor on the left **(2)**	Text area where each memory line contains an instruction or numeric cell	Represents main memory, where program instructions and data live in the same address space
Variable table (X, Y, Z, W, T0/T1...) **(3)**	Editable values used by the current program	Represents data stored in RAM cells; T variables are temporary memory locations
Add T variable button **(4)**	Creates more temporary variables	Extends the available memory cells for intermediate storage
IR (Instruction Register) **(5)**	Shows the current decoded instruction and operand	Holds the instruction currently being executed
Decoder **(6)**	Indicates the control unit stage that interprets the current instruction	Represents the control unit that decides how hardware reacts to the opcode
ALU **(7)**	Shows the arithmetic operation and operands currently in use	Executes arithmetic and logic operations
ACC (Accumulator) **(8)**	Stores the current working value	Central working register used by this machine for arithmetic
PC (Program Counter) **(9)**	Shows the address of the next instruction	Points to the next memory location to fetch
Data bus **(10)**	Visual connection between memory, IR, ALU, and ACC	Carries data and instructions between components

Address bus **(11)**	Visual connection used with the PC and RAM	Carries memory addresses
RAM block on the diagram **(12)**	Abstract diagrammatic memory block	Highlights that both code and data share the same memory system
Execution controls **(13)**	Run loop, Single step, Single iteration, Pause, Stop	Let the user observe the fetch-decode-execute cycle

Correlation with the Von Neumann model

The simulator follows the classic Von Neumann idea that instructions and data are stored in the same memory. The program text written in the RAM editor and the variables on the left are both part of the machine memory. The PC selects the next instruction, the IR stores the fetched instruction, the Decoder interprets it, and the ALU plus ACC carry out the computation. The buses drawn on the right make the data flow explicit, which is useful for understanding the fetch-decode-execute cycle rather than only seeing final numeric results.

2. EXERCISE 2

Exercise 2*: Exploring Pre-existing Programs*

The vnmsim sample library contains 11 example programs. Most of them work correctly and are useful to study how a very small instruction set can still implement non-trivial behavior. The only problematic built-in sample is the one titled Modulo, whose code does not compute a modulo and instead behaves like a comparison routine.

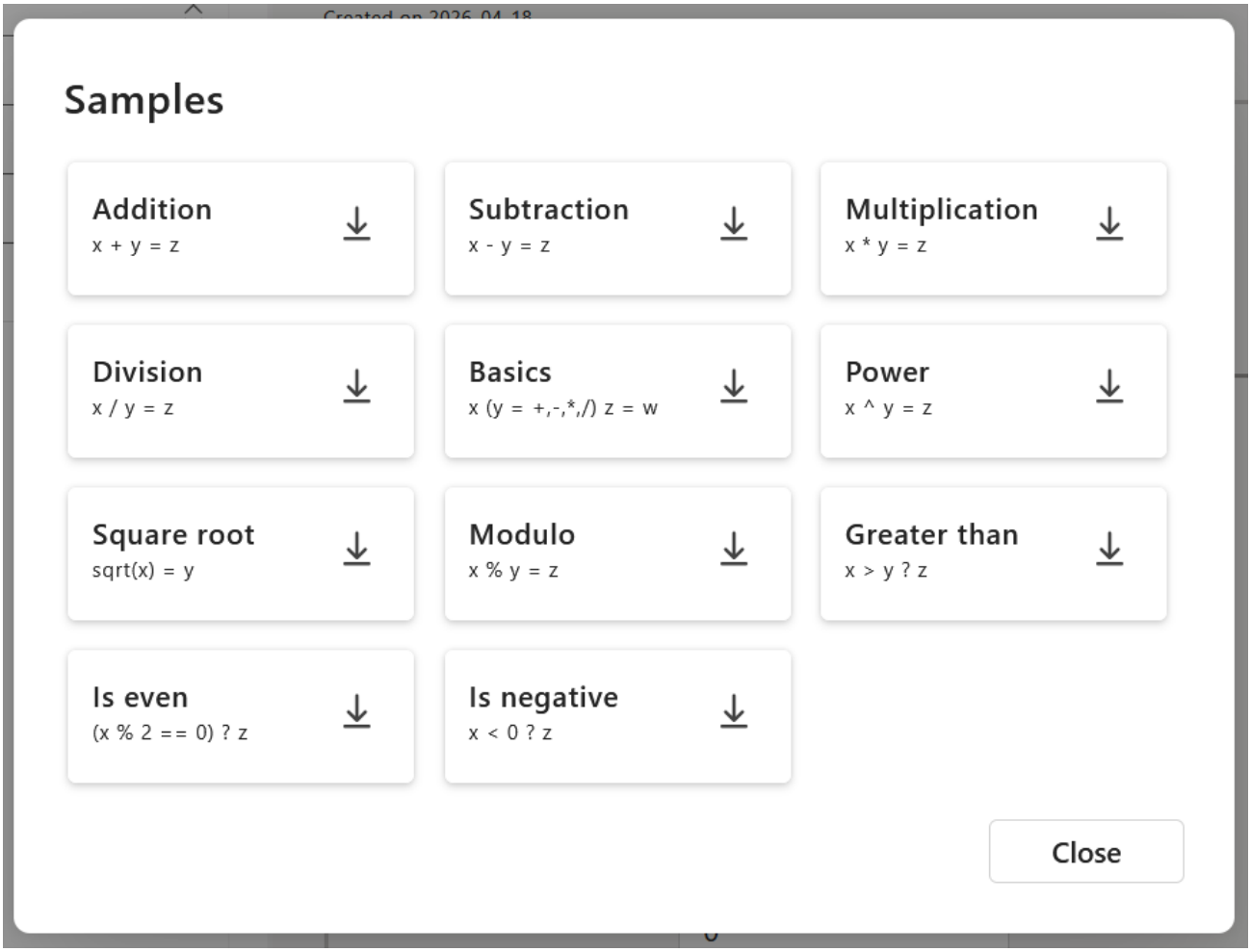

Fig. 2 Sample programs

Sample programs analysis

Sample	Purpose	Initial state	Final state	Works as intended?	Explanation
Addition	Compute X + Y = Z	X=2, Y=2, Z=0, W=0	Z=4	Yes	LOD X loads the accumulator, ADD Y adds the second operand, and STO Z saves the result.
Subtraction	Compute X - Y = Z	X=3, Y=2, Z=0, W=0	Z=1	Yes	Same structure as the addition sample, but using SUB Y.

Multiplication	Compute X * Y = Z	X=6, Y=7, Z=0, W=0	Z=42	Yes	Shows direct use of the MUL instruction with no branching.
Division	Compute X / Y = Z	X=12, Y=3, Z=0, W=0	Z=4	Yes	Minimal arithmetic example for DIV.
Basics	Compute X[Y]Z = W, where Y selects +, -, *, or /	X=3, Y=0, Z=2, W=0	W=5	Yes	The program repeatedly subtracts 1 from Y to detect whether the selected operation is 0, 1, 2, or 3, then jumps to the correct arithmetic branch.
Power	Compute X^Y = Z	X=3, Y=2, Z=0, W=0	Y=0, Z=9	Yes	Initializes Z to 1, then loops: multiply Z by X, decrement Y, repeat until Y=0.
Square root	Approximate sqrt(X) = Y	X=25, Y=0, Z=0, W=0	Y=5, Z=0	Yes	Uses an iterative method: initialize Y, keep a loop counter in Z, then update Y = (Y + X / Y) / 2 until the counter reaches zero.
Is even	Decide whether X is even and store the result in Z	X=4, Y=0, Z=0, W=0	Y=4, Z=1	Yes	Divides by two, reconstructs 2 * floor(X/2) into Y, then checks whether X - Y = 0.
Greater than	Decide whether X > Y and store the	X=3, Y=2, Z=0, W=0, T1..T4=0	Z=1	Yes	Because the machine has only JMZ, the sample simulates comparison by copying positive and negative counters into temporary variables and

	Boolean result in Z				decrementing them until one side reaches zero first.
Is negative	Decide whether X < 0 and store the Boolean result in Z	X=-3, Y=0, Z=0, W=0, T1=0, T2=0	Z=1, T2=3	Yes	The program derives helper counters from X, then uses repeated subtraction and JMZ to distinguish negative from non-negative input.
Modulo	Title suggests X % Y = Z	X=3, Y=2, Z=0, W=0	X=0, Y=0, Z=1	No	The code repeatedly decrements X and Y together and sets Z to either 0 or 1. It behaves like a comparison routine, not a modulo implementation. The sample appears mislabeled or incomplete.

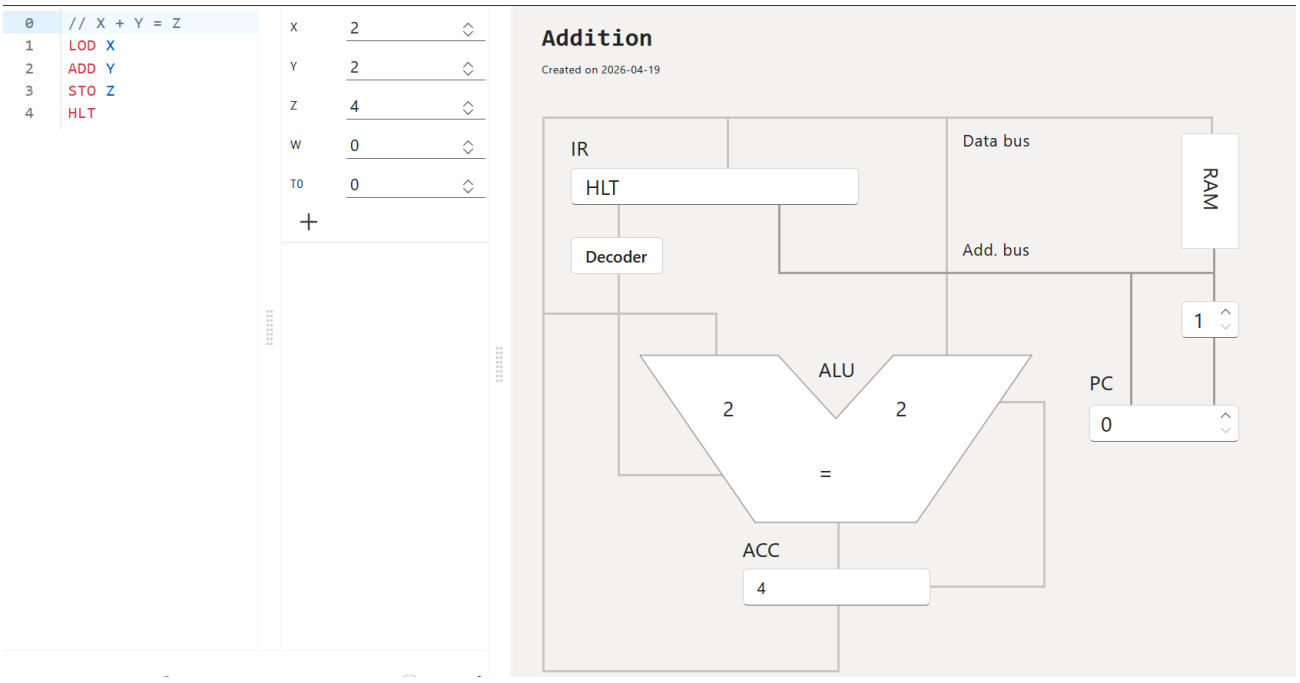

Fig.3 Addition

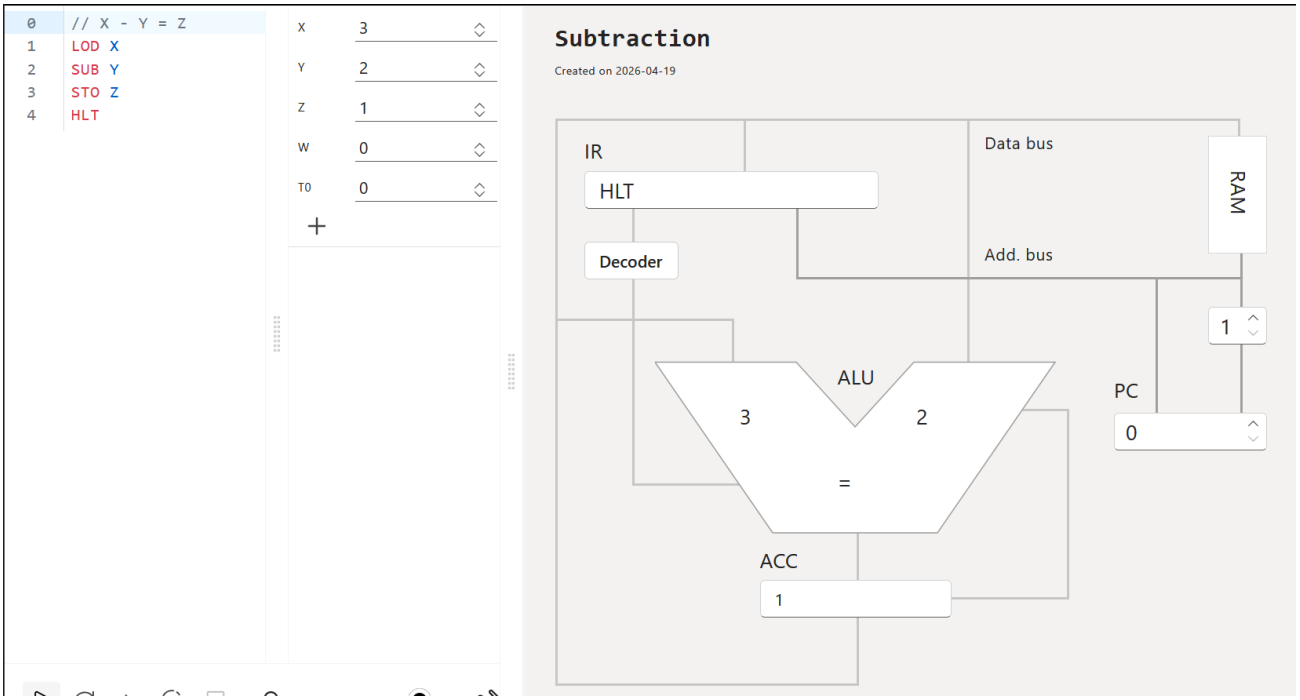

Fig. 4 Subtraction

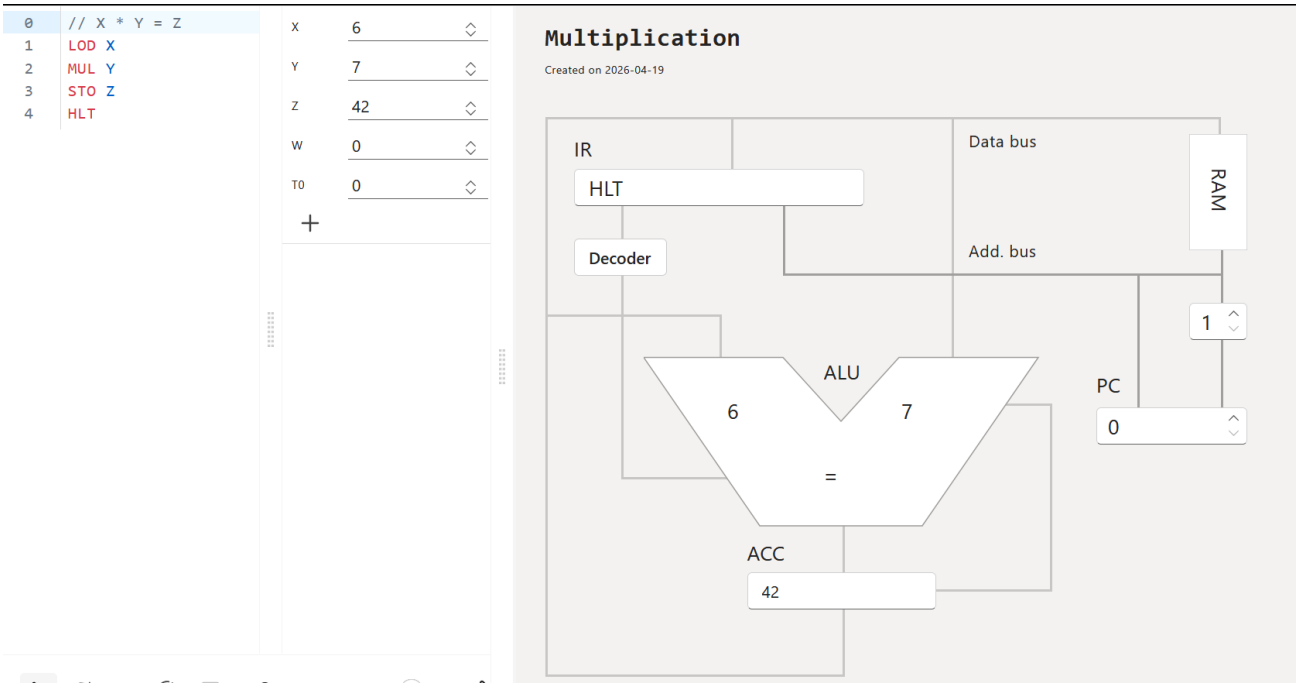

Fig.5 Multiplication

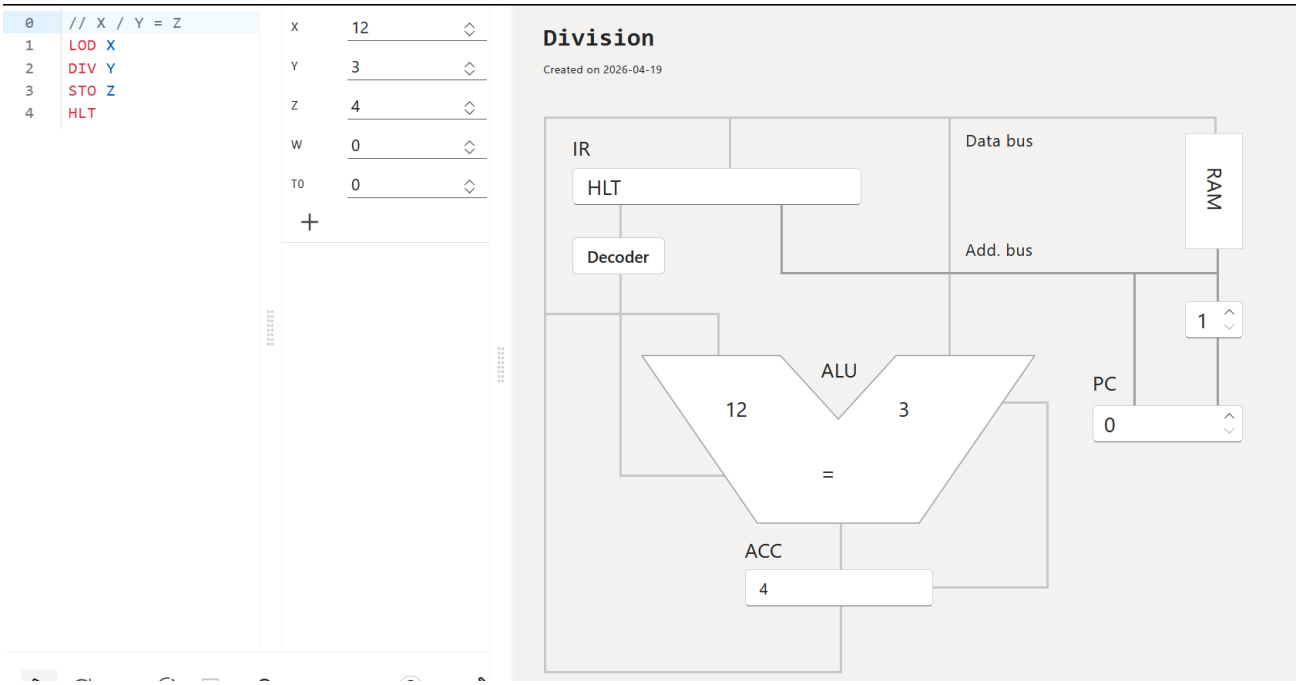

Fig. 6 Division

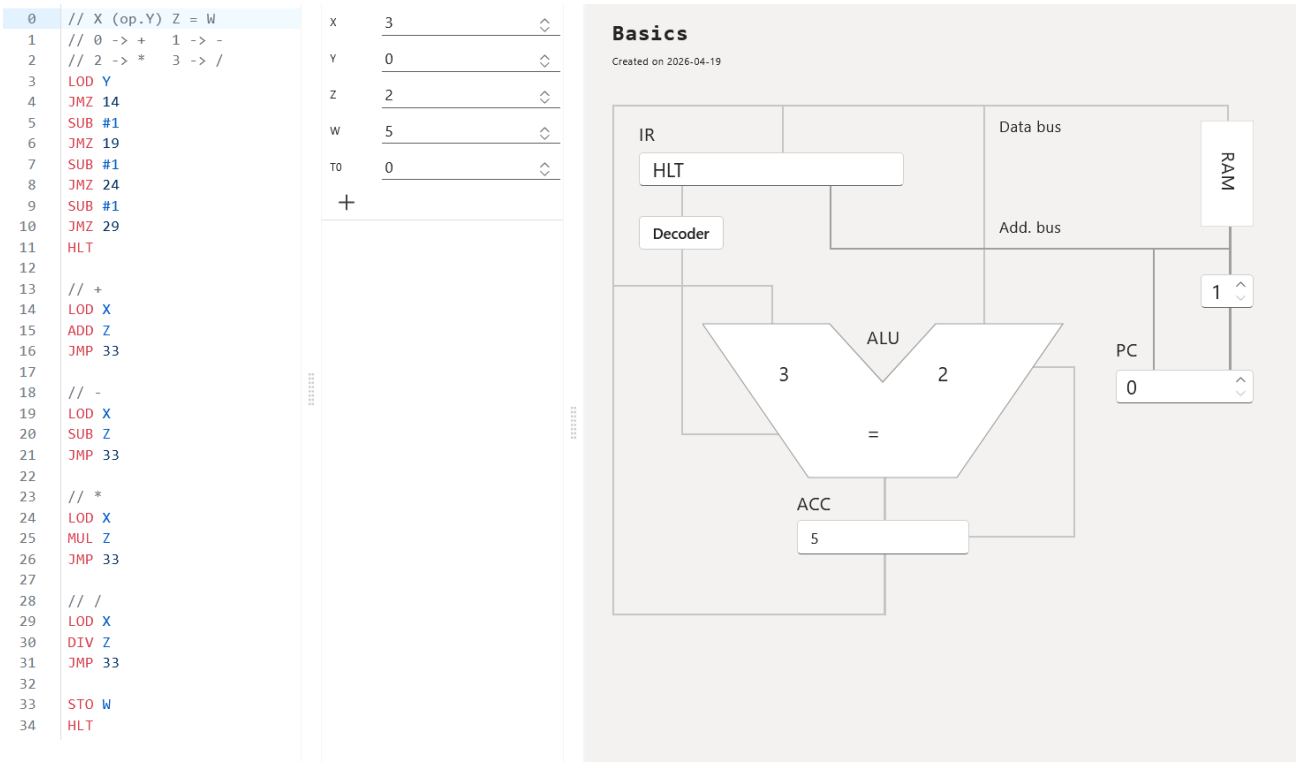

Fig. 7 Basics

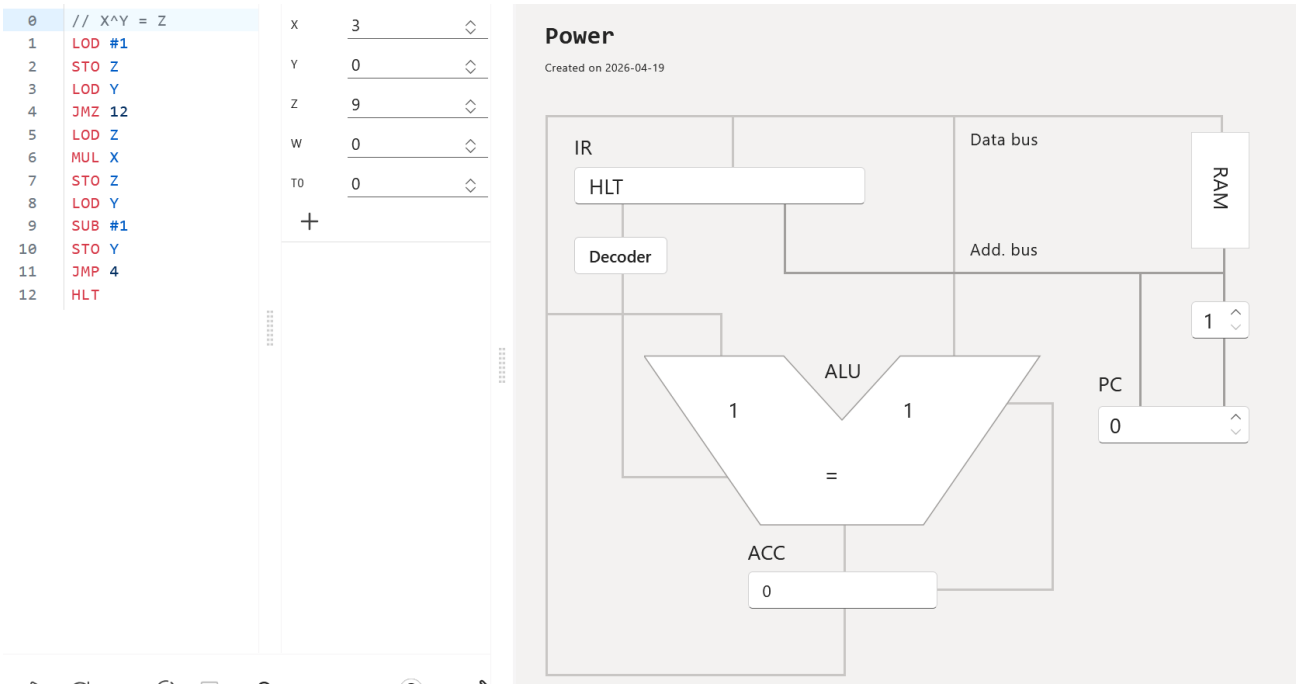

Fig. 8 Power

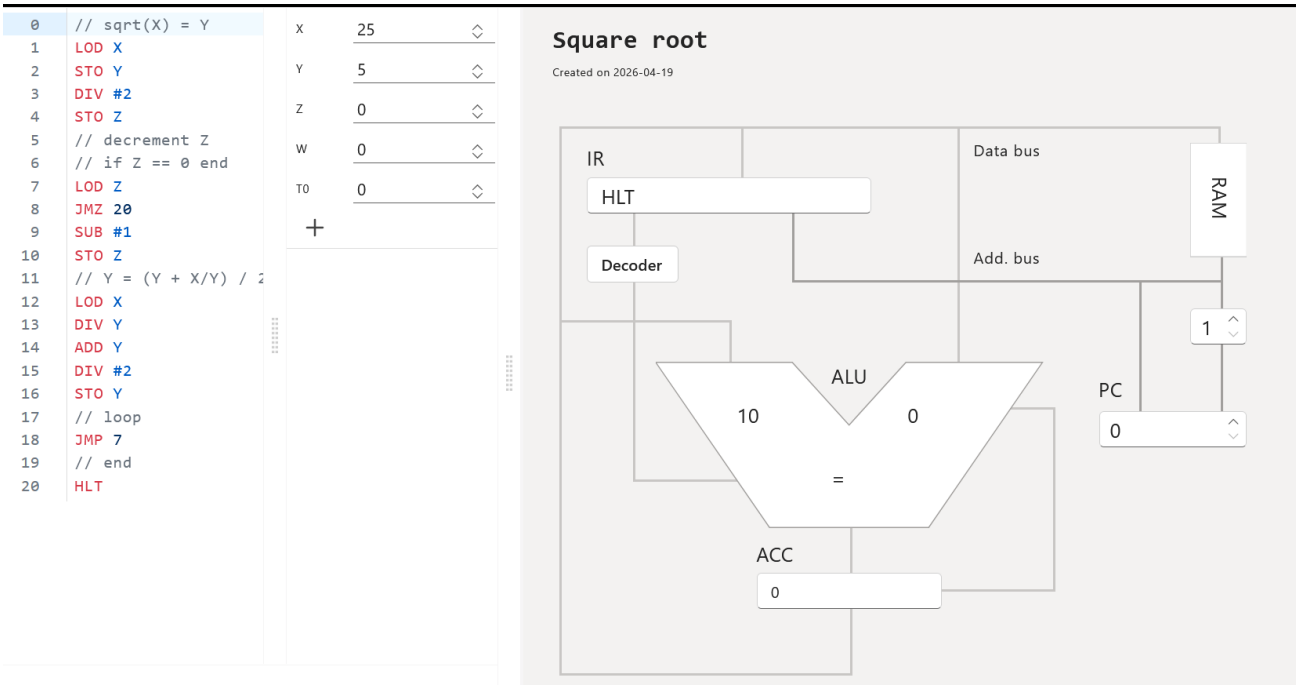

Fig. 9 Square root

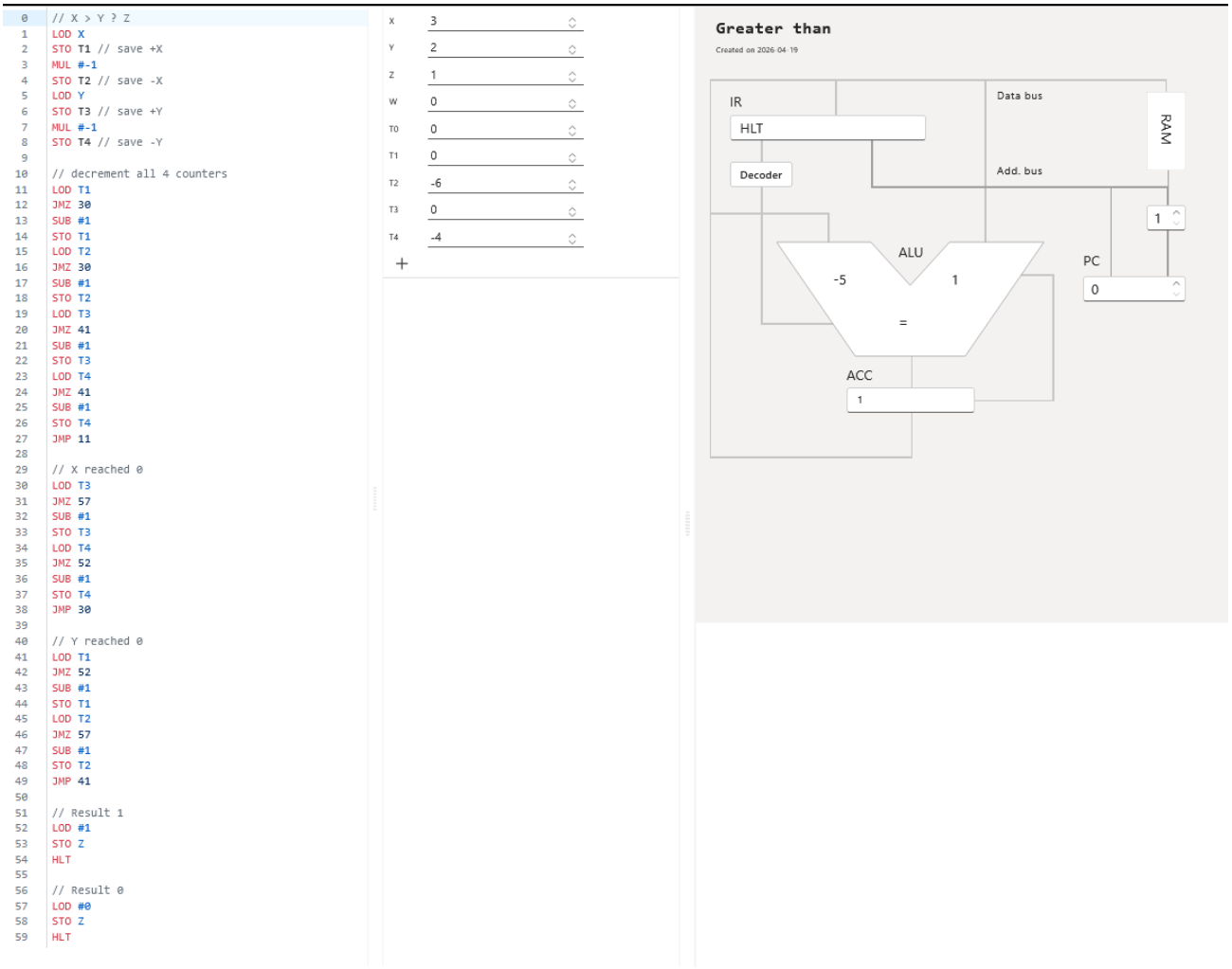

Fig. 10 Greather than

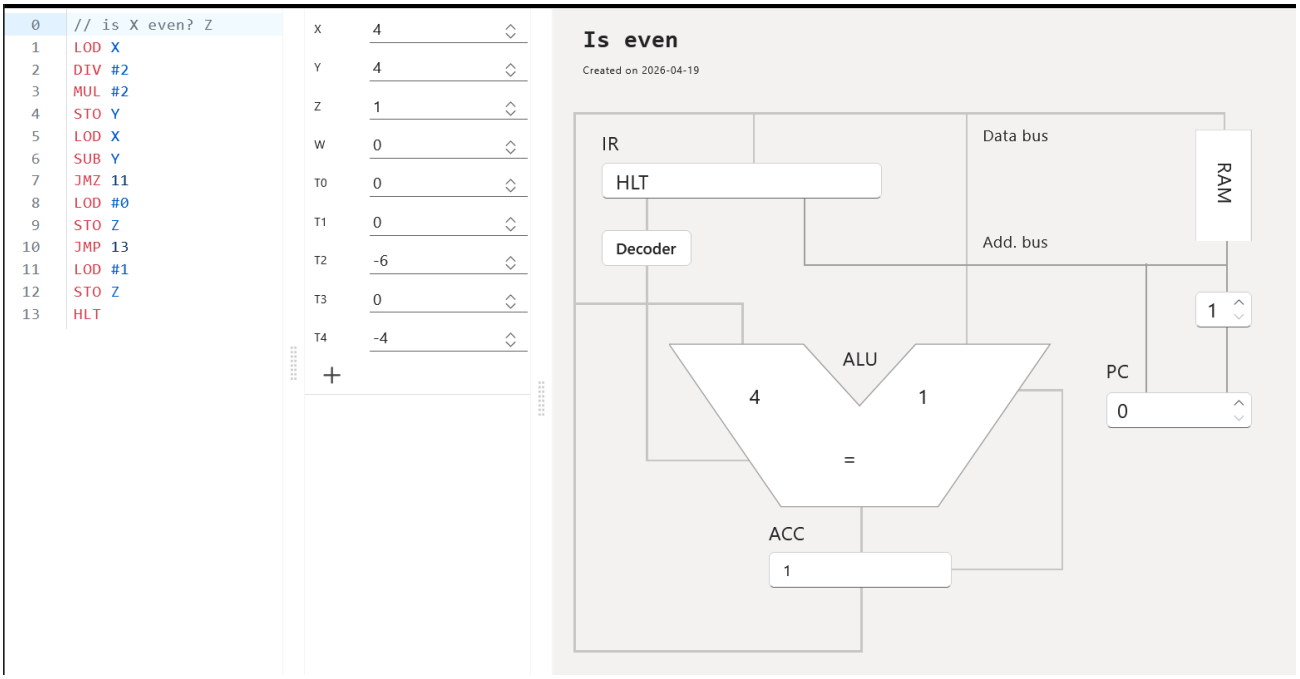

Fig. 11 Is even

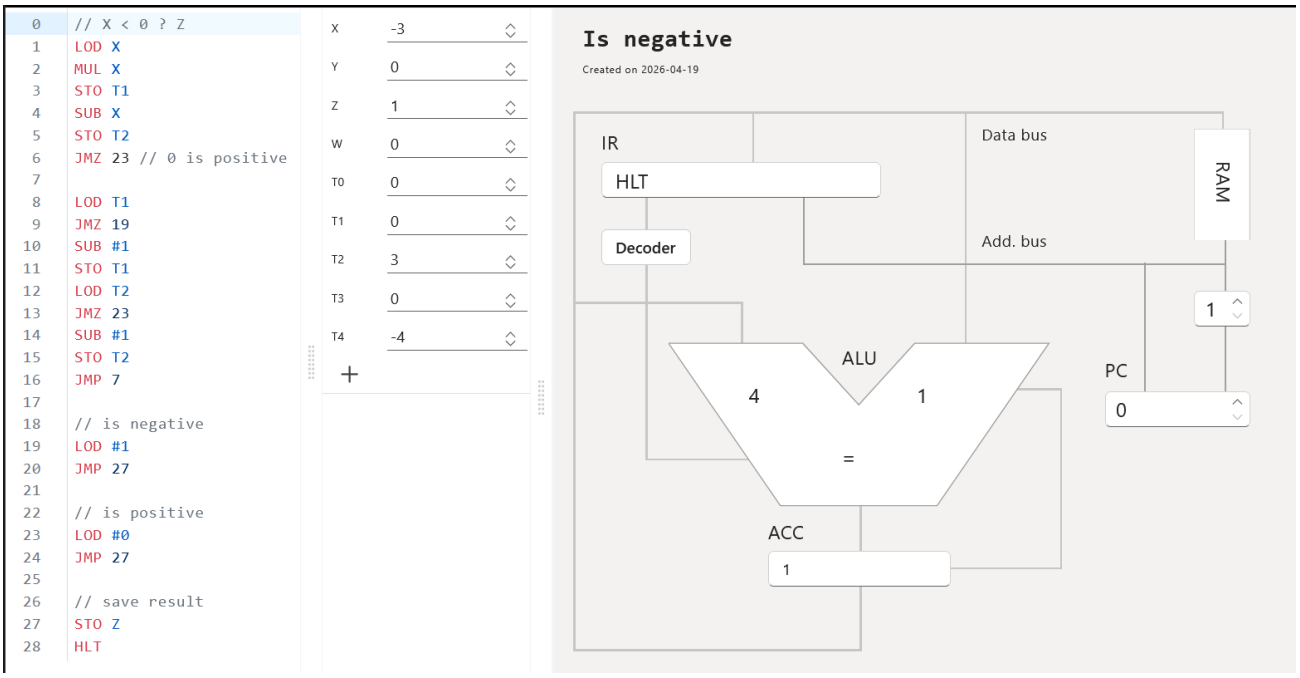

Fig. 12 Is negative

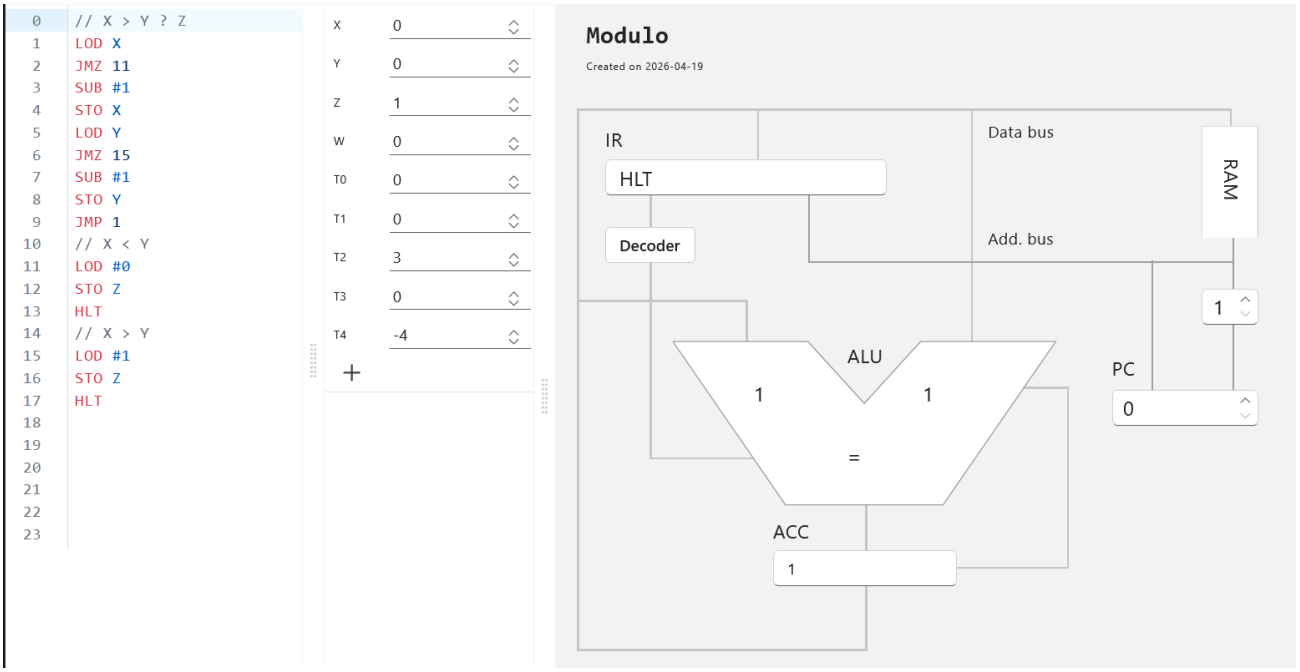

Fig. 13 Modulo

General observations about the samples

1. Straight arithmetic operations are very compact: LOD, one ALU instruction, STO, HLT.
2. Branching is limited to JMZ, so every non-trivial decision is reduced to checking whether the accumulator is exactly zero.
3. Because there is no indexed addressing over T1..T10, more advanced programs often rely on repeated subtraction and extra temporary variables.
4. The sample set is useful because it shows both the strength and the limitation of accumulator-based programming: simple math is easy, but comparisons and selectors require creative control flow.

3. EXERCISE 3

Exercise 3: *Conditional swapping of two numbers*

Requested behavior

- Read two numbers from T1 and T2.
- Read control value X.
- If X = 0, do not swap.
- If X = 1, swap T1 and T2.

- For any other value, stop without changing the data.

Program design

The simulator only provides a zero-test jump (JMZ). Because of that, the cleanest way to detect X = 1 is:

1) Load X.
2) If it is already zero, stop immediately.
3) Subtract 1.
4) If the result is zero, then the original value was 1, so run the swap.
5) Otherwise halt.

The swap itself needs one temporary memory location, so T3 is used as auxiliary storage.

Program

```
LOD X // load the control value
JMZ 11 // X = 0 -> keep T1 and T2 unchanged
SUB #1 // ACC = X - 1
JMZ 5 // X = 1 -> execute the swap
HLT
LOD T1 // save the original first number
STO T3 // use T3 as temporary storage
LOD T2 // load the second number
STO T1 // copy T2 into T1
LOD T3 // recover the original T1 value
STO T2 // copy the original T1 into T2
HLT
```

Example test cases

Case	**Initial values**	**Expected final values**	**Result**
Keep values	X=0, T0=0, T1=8, T2=3, T3=0	T0=0, T1=8, T2=3, T3=0	No swap

Swap values	X=1, T0=0, T1=8, T2=3, T3=0	T0=0, T1=3, T2=8, T3=8	Swap performed
Invalid selector	X=2, T0=0, T1=8, T2=3, T3=0	T0=0, T1=8, T2=3, T3=0	Safe halt

Challenges and solution choices

The main limitation was the absence of a direct instruction such as JNZ, JGT, or CMP. The solution avoids unnecessary complexity by using two zero-tests in sequence. This is reliable, short, and easy to explain.

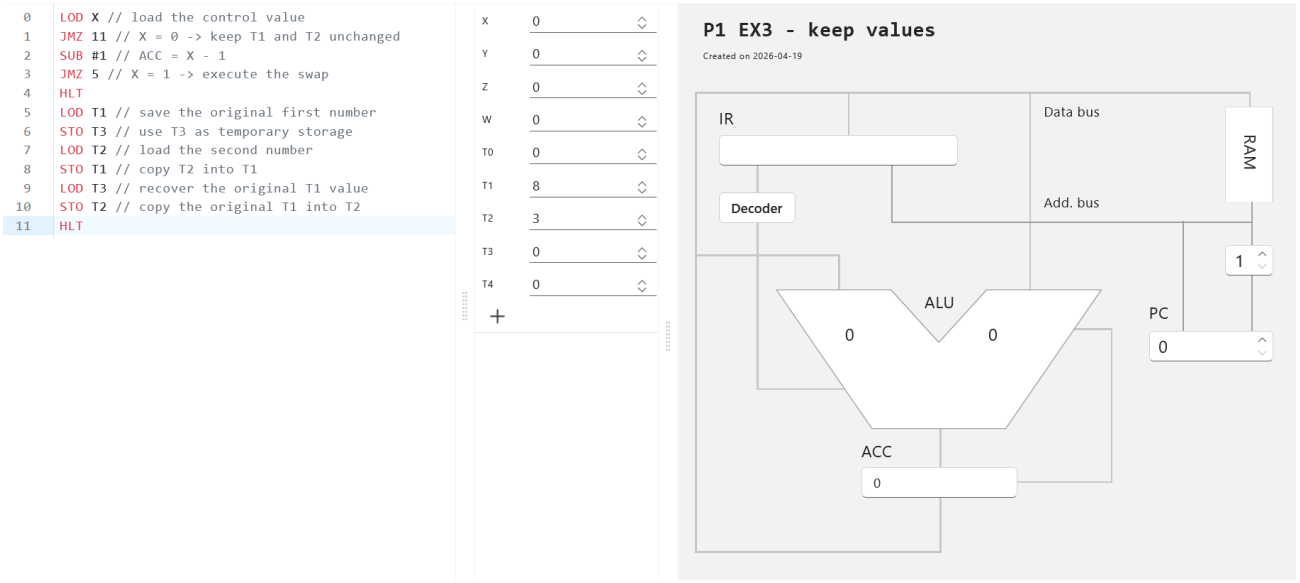

Fig.14 Before run (keep values)

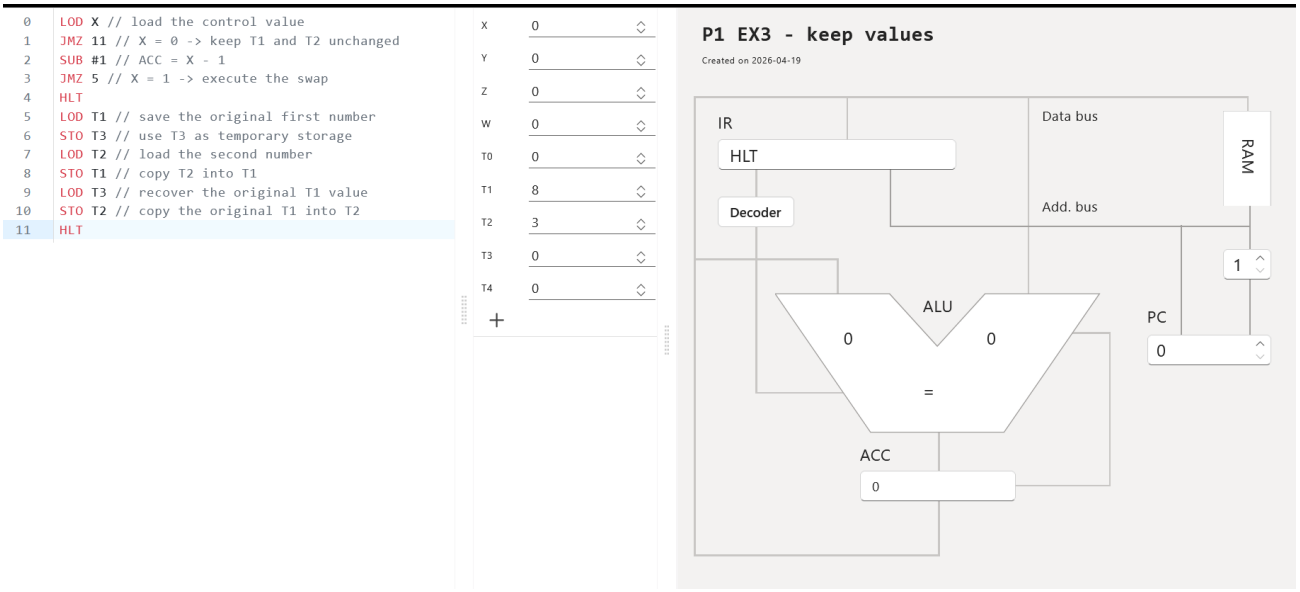

Fig. 15 After run (keep values)

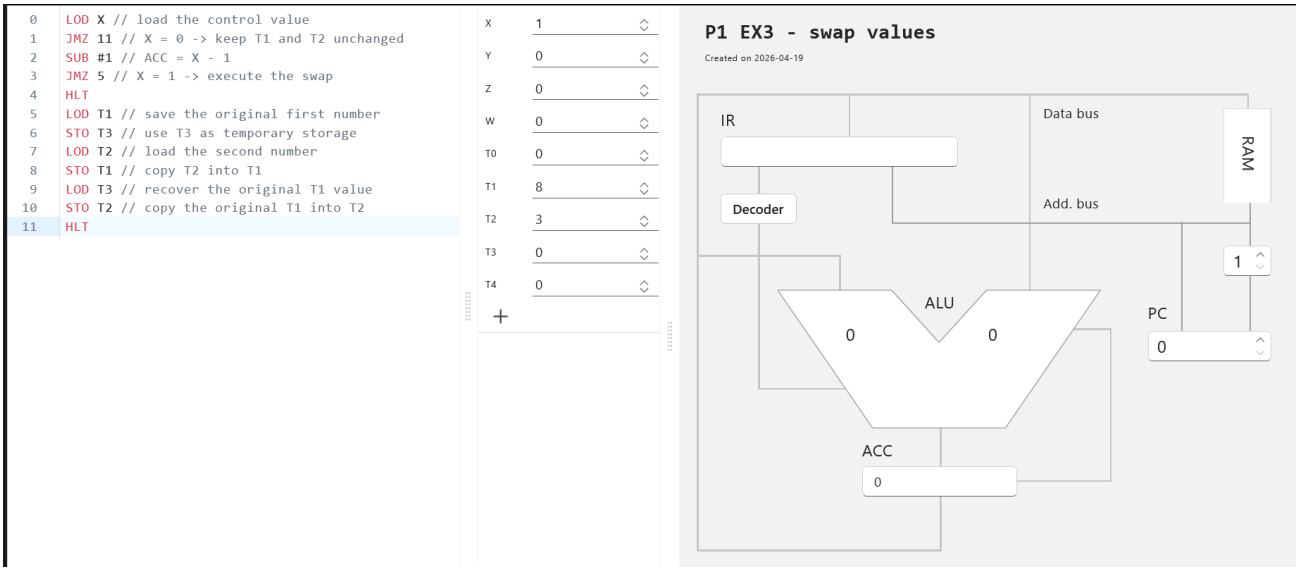

Fig. 16 Before run (swap values)

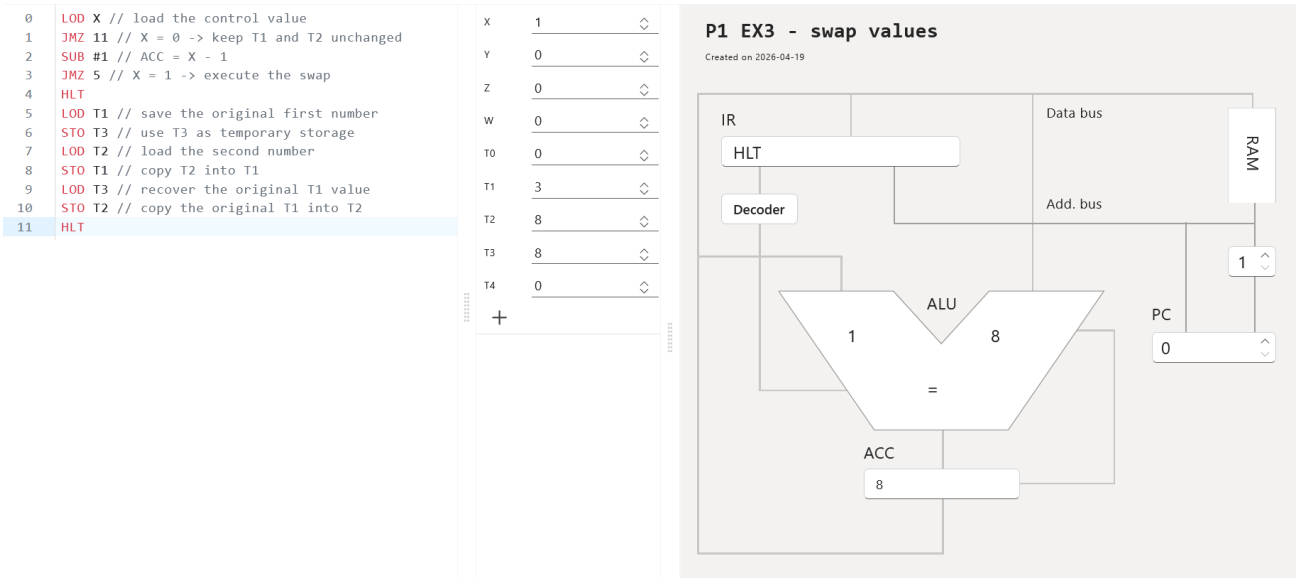

Fig. 17 After run (swap values)

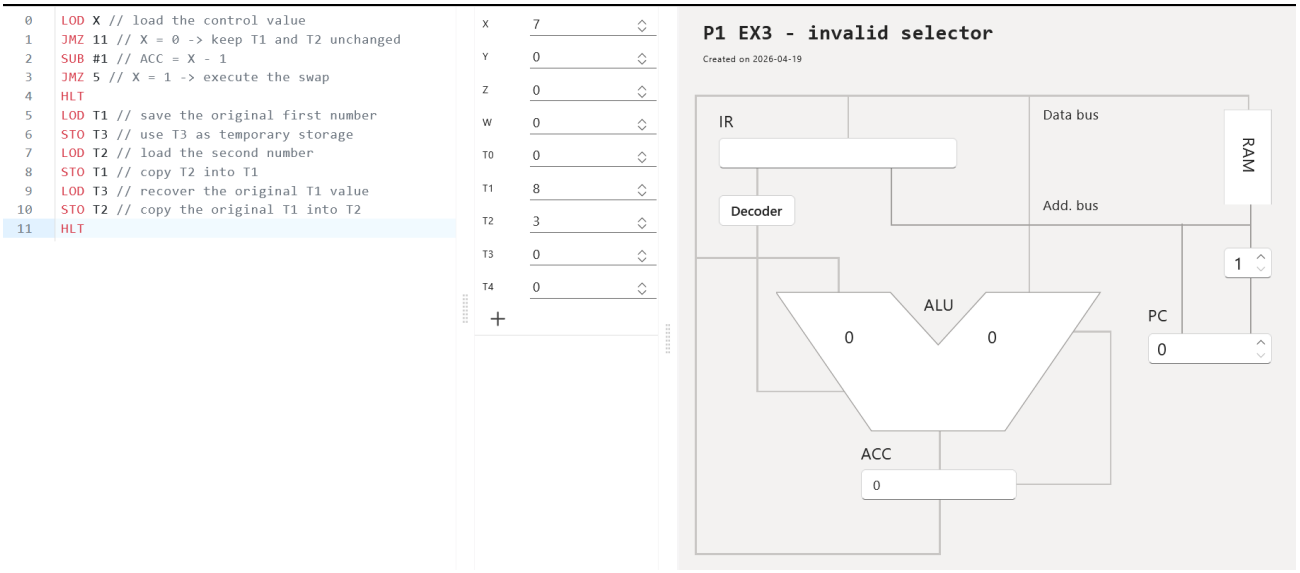

Fig. 18 Invalid selector before run

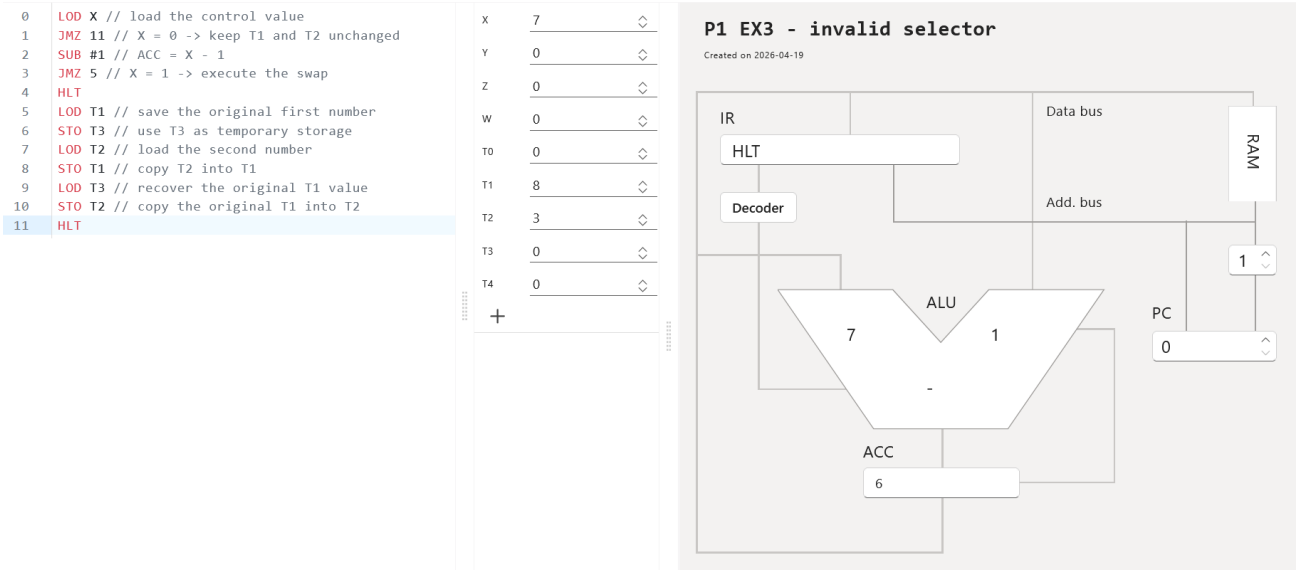

Fig. 19 Invalid selector after run

4. EXERCISE 4

Exercise 4: *Conditional Addition of Numbers from an Array*

Requested behavior

- Store ten numbers in T1..T10.
- If X = 0, sum all ten values.
- If X = 1, sum the first five values.
- If X = 2, sum the last five values.
- If X has any other value, do nothing.

Program design

The result is stored in W.

Because vnmsim does not support indexed access to T1..T10, the most robust implementation does not use a counter-driven loop. Although JMZ can create loops (as seen in the Power and Square Root samples), vnmsim has no indirect addressing, so there is no way to access T1..T10 by a variable index. The additions are therefore unrolled explicitly. This matches the simulator's

limitations and keeps the control flow easy to verify.

The selector logic is:

1) If X = 0, jump to the branch that sums T1..T10.

2) Otherwise subtract 1 and check whether the original value was 1.

3) Otherwise subtract 1 again and check whether the original value was 2.

4) If none of those branches match, halt without changing W.

Program code

```
LOD X // load the selector value
JMZ 7 // X = 0 -> sum all ten numbers
SUB #1 // ACC = X - 1
JMZ 19 // X = 1 -> sum the first five numbers
SUB #1 // ACC = X - 2
JMZ 26 // X = 2 -> sum the last five numbers
HLT
LOD T1 // start full-array sum
ADD T2 // add T2
ADD T3 // add T3
ADD T4 // add T4
ADD T5 // add T5
ADD T6 // add T6
ADD T7 // add T7
ADD T8 // add T8
ADD T9 // add T9
ADD T10 // add T10
STO W // store the total sum in W
HLT
LOD T1 // start first-half sum
ADD T2 // add T2
ADD T3 // add T3
ADD T4 // add T4
ADD T5 // add T5
STO W // store the sum of T1..T5 in W
HLT
LOD T6 // start second-half sum
ADD T7 // add T7
ADD T8 // add T8
ADD T9 // add T9
ADD T10 // add T10
STO W // store the sum of T6..T10 in W
HLT
```

Example test cases

For the following cases the input array is:

T1=1, T2=2, T3=3, T4=4, T5=5, T6=6, T7=7, T8=8, T9=9, T10=10

Selector X	Required behavior	Expected result in W
0	Sum all ten numbers	55
1	Sum first five numbers	15
2	Sum last five numbers	40
9	Do nothing	0

Challenges and solution choices

The largest difficulty was the lack of indirect addressing. A normal loop over T1..T10 driven by an index variable is not possible in this simulator. JMZ jumps can create loops, but they can only reference fixed, named cells. Because there is no indirect addressing, there is no way to write LOD T[counter] where the address changes each iteration. The answer is therefore an unrolled program with three clearly separated branches. It is longer, but it is correct and easy to debug.

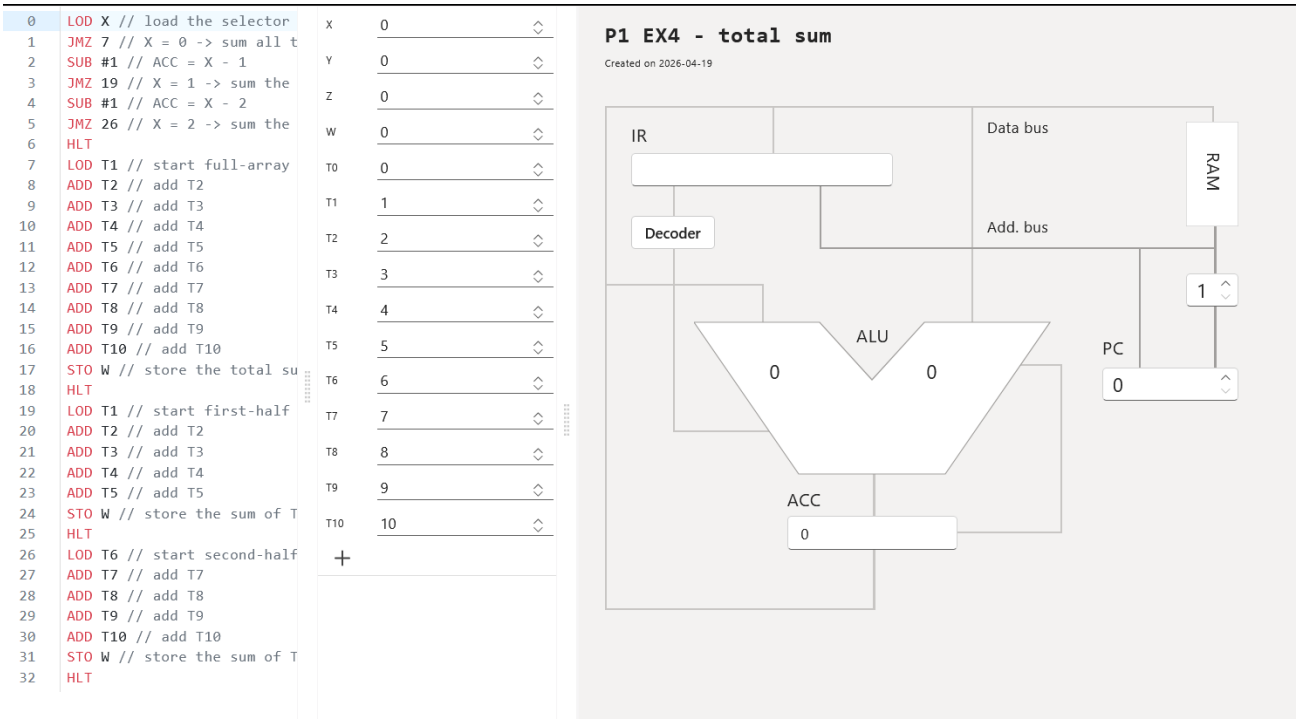

Fig. 20 Total sum (before run)

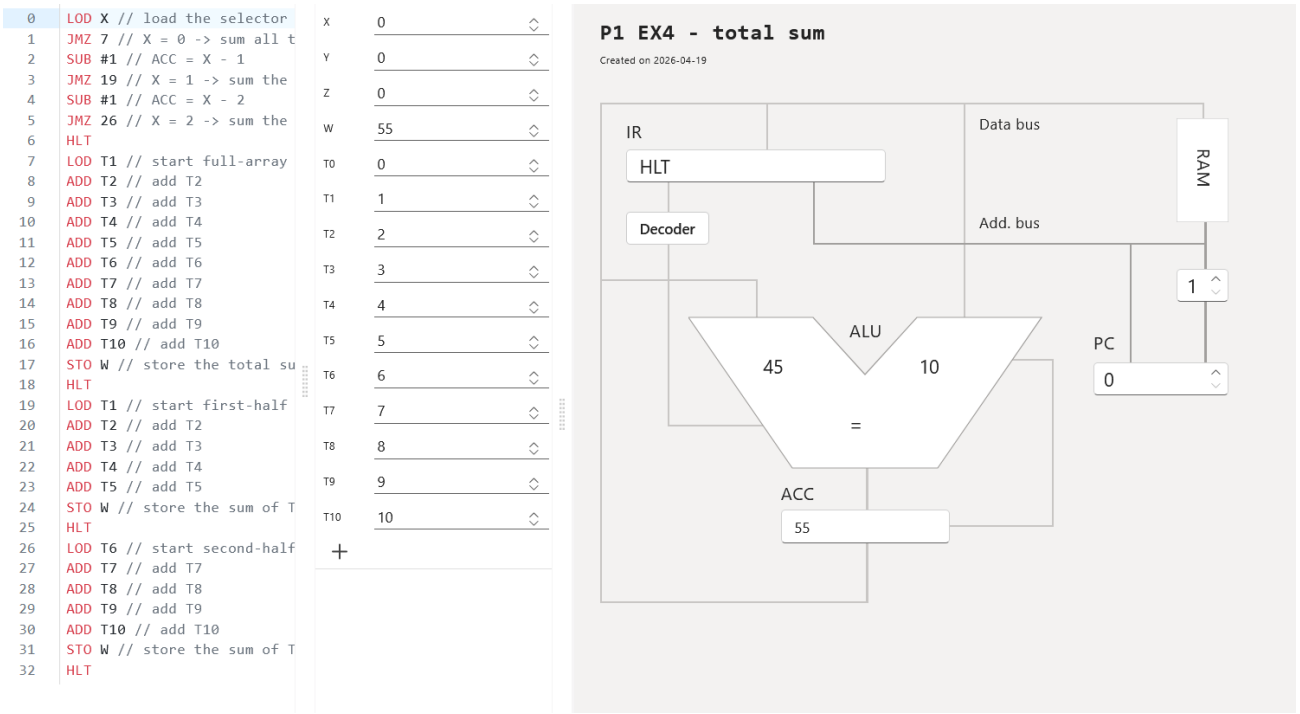

Fig. 21 Total sum (after run)

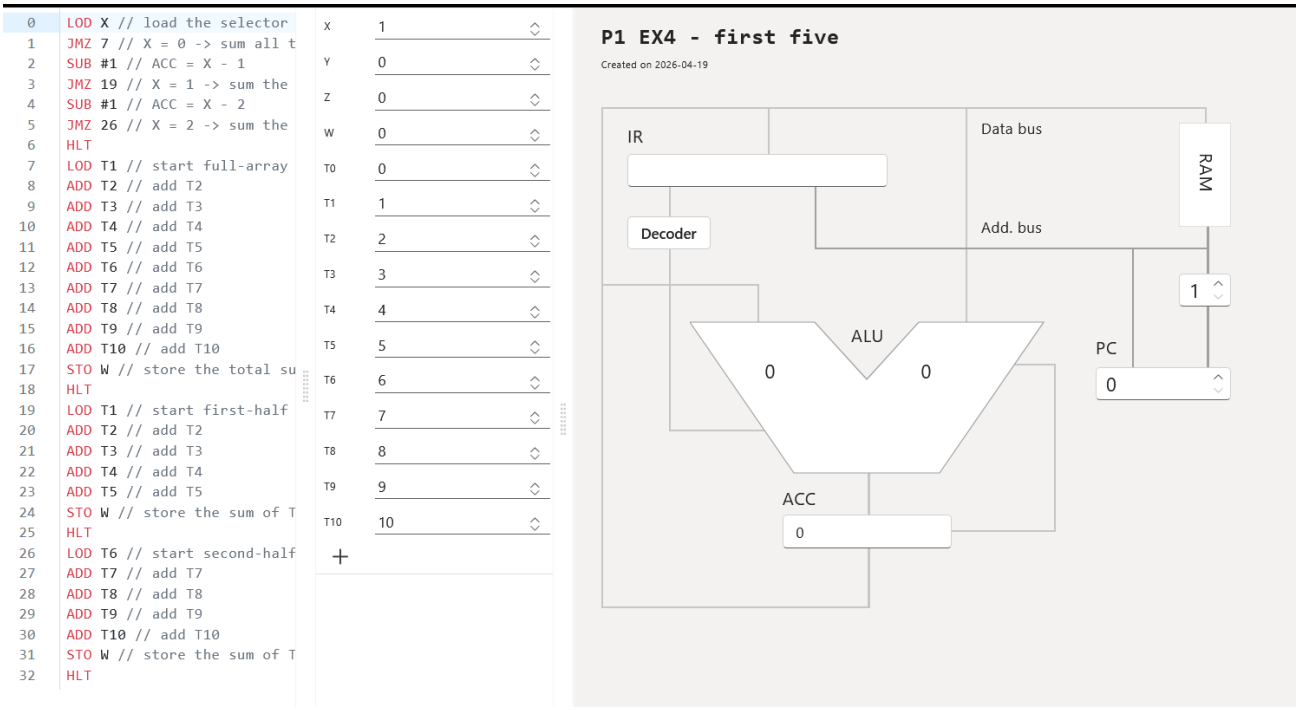

Fig. 22 Sum first five (before run)

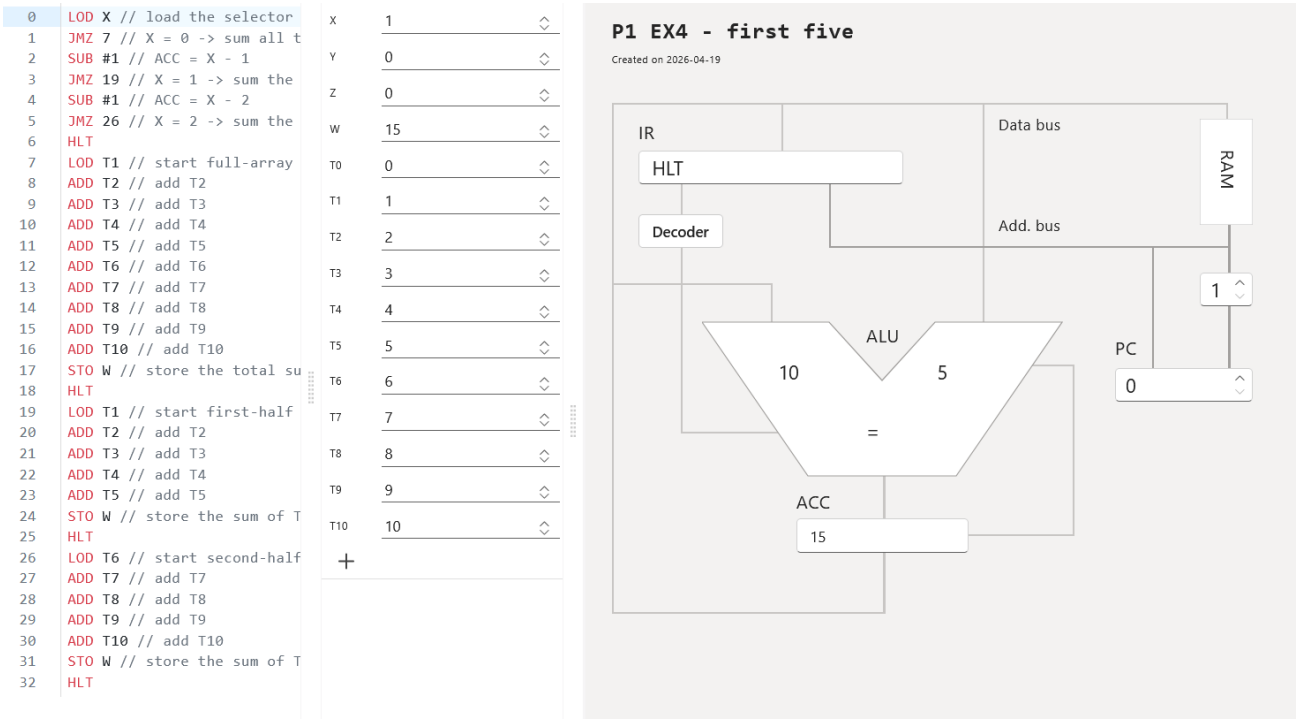

Fig. 23 Sum first five (after run)

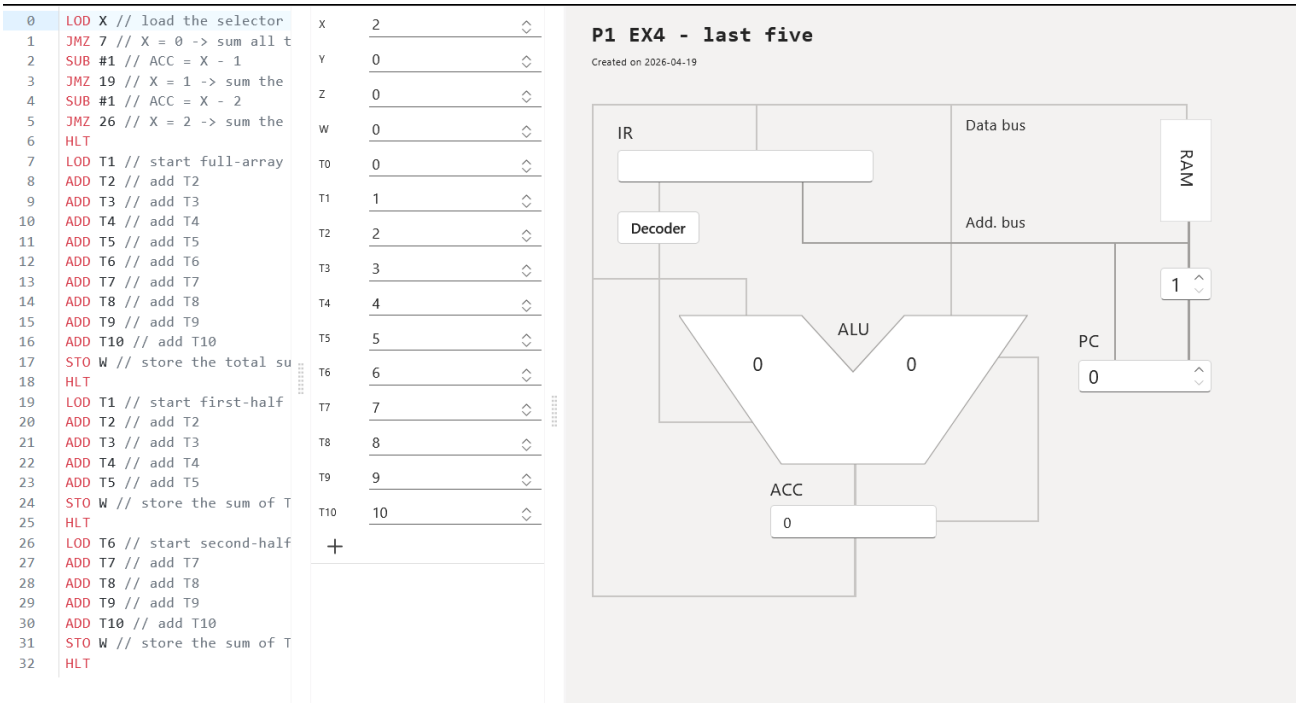

Fig. 24 Sum last five (before run)

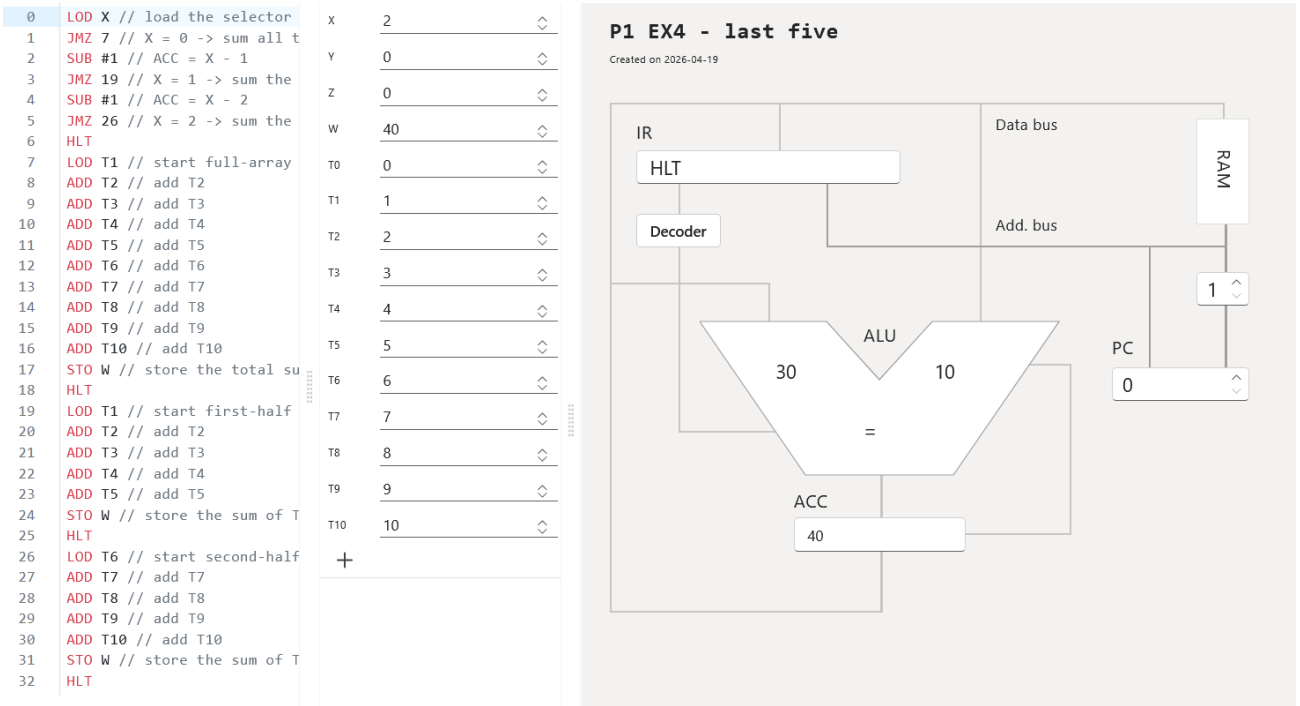

Fig. 25 Sum last five (after run)

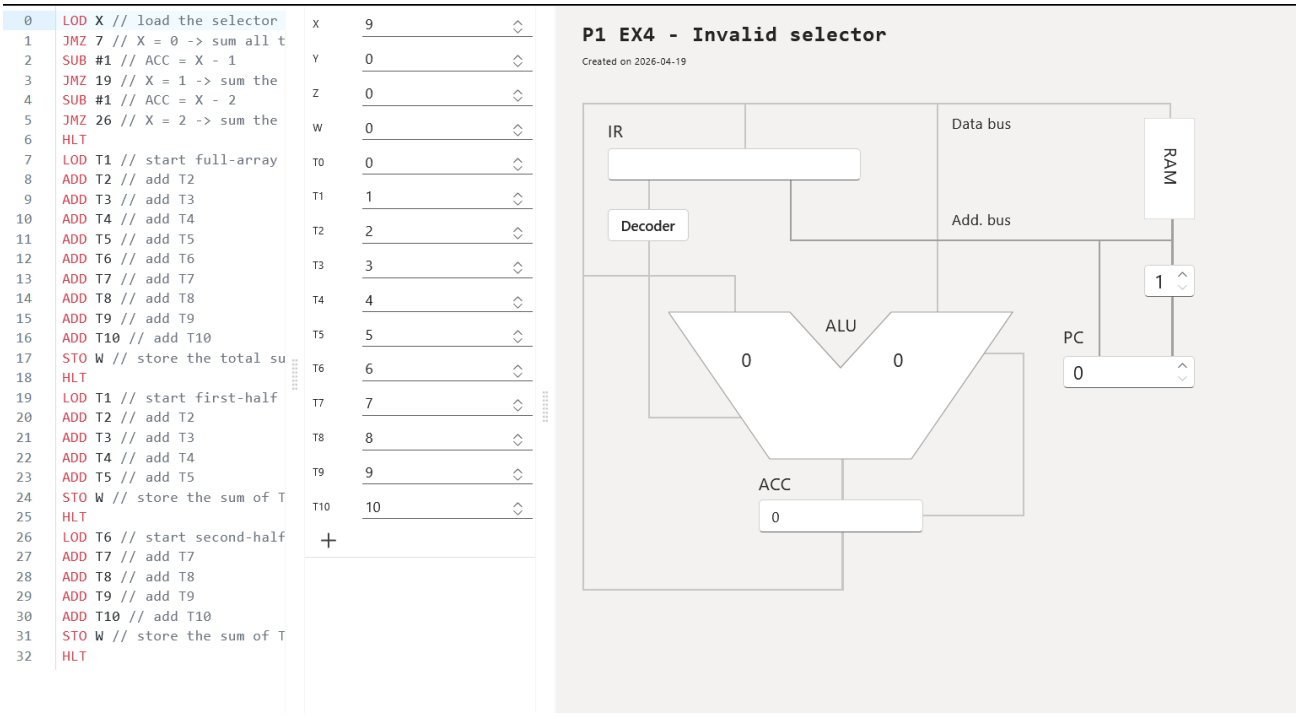

Fig. 26 Invalid selector (before run)

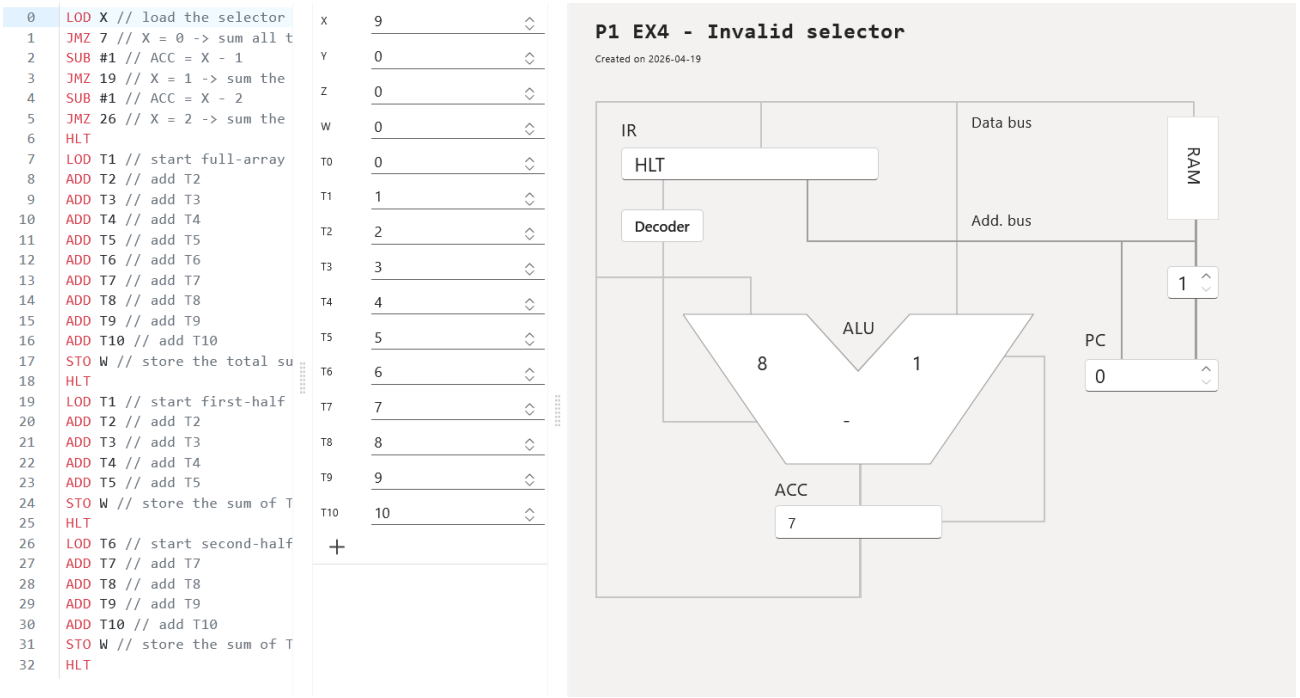

Fig. 27 Invalid selector (after run)

5. CONCLUSION

This practical work shows that even a very small Von Neumann-style machine can solve meaningful problems, but simple high-level ideas such as comparison, selection, and array processing become much more manual when the instruction set only provides an accumulator and a jump-if-zero operation. That is what makes vnmsim useful for learning: it exposes the control flow and data movement that higher-level languages usually hide.